

M-Agent 总体设计架构

蓝色章节条用于定位结构；黄色提示框表示结论、边界或风险；表格中的加粗项是当前决策重点。

源文档：overall-design-architecture.md · 状态日期：2026-07-29

Overall Design Architecture

本文是 M-Agent Runtime 演进过程中的设计总线。它规定迁移前后必须保持稳定的领域结构、职责边界和运行逻辑，不限定内部使用何种框架、存储方式或代码组织。

1. 三大结构性功能的设计意义

1.1 Transaction 事务线与 Scene 情景线

Transaction 与 Scene 描述系统运行中的两个不同维度。

Transaction 是隶属于某个 conversation、围绕同一任务稳定存在的交互记录。一次 transaction 可以跨越多轮交互；它在内容上表现为工作记忆（WM），在状态上表现为任务状态。它主要回答：

- 当前处理的是哪一件事；
- 这件事已经进行到哪里；
- 后续能否自主继续；
- 是否正在等待用户、时间或其他条件；
- 任务是否已经完成；
- 将来恢复时应继续使用哪些任务上下文。

Scene 是 conversation 级别连续发展的情景线，用于描述整个 conversation 中实际发生过的事情。一个 conversation 可以同时或先后存在多个 transaction，但这些 transaction 共同构成一条按实际发生顺序延续的 Scene。它主要回答：

- conversation 中先后发生了哪些事情；
- 用户输入、系统行为、外部结果和回复之间是什么顺序；
- 多个 transaction 的活动如何共同形成完整的交互历史。

Scene 的连续性不是无限堆叠。Flush 是用户可感知的语义边界，用于结束一段连续交互、巩固本段记录，并区分前后两场不再默认连续的会话。Flush 之前和之后发生过的事实仍然可以保留，但后续无来源刺激不应假定能够自然续接已经归档的任务。

两者不能互相替代：

- Transaction 保存某一任务的当前工作状态。
- Scene 保存整个 conversation 的连续经历。
- Transaction 暂停、完成或归档，不应删除已经发生的 Scene。
- Scene 中存在某项记录，也不等于相应 transaction 已经完成。

把两条线分开，能够让多个任务在同一 conversation 中保持各自的身份、WM 和任务状态，同时让用户所经历的整体交互仍然是一条连续、可理解的情景历史。

1.2 Conversation 内的感知刺激池

感知刺激池用于承接一个 conversation 中已经实际到达、正在等待处理的刺激，例如用户消息、通过来源有效性校验的 Execution Feedback、到期的 Scheduled Plan，以及未来新增的其他刺激。

它把“刺激何时到达”与“系统何时处理”分开，使系统能够：

- 同时存放多个不同类别的刺激；
- 保持每个刺激原有的身份、内容、来源和优先级；
- 在同一 conversation 内按照优先级有序处理；
- 在相同优先级下保持稳定的接纳先后顺序；
- 始终逐个消费，避免同一 conversation 中多个刺激同时争抢任务状态；
- 在消费期间继续接纳后来到达的刺激；
- 将思考过程与异步工具执行解耦，不因等待工具结果而阻塞后续刺激。

刺激池只属于其所在的 conversation。不同 conversation 的刺激不在同一个排序空间中，也不需要互相比较优先级。

尚未发生的未来计划不属于当前刺激池。Scheduled Plan 只有在到达激活时间、真正生成刺激以后，才进入所属 conversation 的刺激池。

刺激池只负责保存和安排已经到达的刺激，不负责判断刺激属于哪个 transaction，也不负责创建、恢复或修改 transaction。

1.3 无来源刺激的 Transaction 归因

刺激离开感知池以后，系统必须明确它接下来属于哪一个 transaction。

部分刺激天然带有来源。例如 Feedback、Scheduled Plan，以及未来可能出现的其他有来源刺激，会携带明确的 transaction 身份。这类刺激不需要语义匹配，但仍必须通过与其来源类型相对应的有效性校验，才能回到指定 transaction。

无来源刺激没有明确的 transaction 身份，系统需要判断：

- 它是否在继续某个已经存在的任务；
- 还是在表达一件新的事情，需要新建 transaction。

归因机制用于维持任务边界：

- 避免每条新刺激都创建 transaction，导致同一任务被切碎；
- 避免把无关刺激错误合并，污染已有 transaction 的 WM 和任务状态；
- 让暂停任务和刚刚完成但尚未归档的任务能够通过后续交互自然恢复；
- 保证一个刺激最终只锁定一个 transaction；
- 在无法明确判断时，通过新建 transaction 保持已有任务的隔离性。

归因的基本原则是：错误合并会污染已有任务，其风险高于多建一个 transaction。因此，无法明确归属时应当新建 transaction。

三大结构性功能分别回答三个不同问题：

- 感知刺激池回答：“这个 conversation 接下来处理哪个刺激？”

- Transaction 归因回答：“这个刺激属于哪一件事情？”
- Transaction 与 Scene 回答：“这件事情如何继续，以及整个 conversation 发生了什么？”

1.4 三大功能共同形成的系统骨架

三大功能不是三个彼此独立的模块，而是同一条领域主线上的三个职责边界：

实际到达的刺激

- 由感知入口完成来源有效性校验
- 由感知刺激池决定处理次序
- 由归因逻辑确定 **transaction** 归属
- 由 **Transaction** 推进具体任务
- 由 **Scene** 保留 **conversation** 的连续经历

幂等、恢复、并发控制、工具调用、审计等能力都很重要，但它们是保障这条主线可靠运行的横向约束，不构成第四个结构性功能。

2. 三大功能的设计逻辑

2.1 Transaction 事务线与 Scene 情景线的运行逻辑

2.1.1 Transaction 的基本组成

每个 transaction 都具有稳定且唯一的身份，并始终隶属于一个 conversation。

Transaction 的 WM 保存继续处理该任务所需要的上下文；任务状态表示当前是否可以继续运行、是否正在等待、是否已经完成或是否已经归档。

同一 transaction 即使经历多轮交互、暂停、重新激活或归档恢复，也必须继续使用原 transaction 身份，而不是创建一个外观相似的新 transaction。

Transaction 的存在不以当前轮次或当前执行过程为边界。状态变化不能导致其身份、WM 或任务状态无故丢失；处于 **pause**、**complete** 或 **archive** 的 transaction 必须按照相应规则保持可恢复。

同一 transaction 可以先后经历多个彼此独立的执行批次。Transaction 身份回答“这是哪一项长期任务”，执行批次身份回答“当前结果来自该任务的哪一次有效激活”。新建任务，或者从已经结束、失效的批次中恢复任务时，应开启新的执行批次；本批次产生的工具委托与 Feedback 必须保持相应的批次关联。为了等待本批次 Feedback 而暂时停止思考，不会另开执行批次。

2.1.2 Transaction 的四种状态

Transaction 具有四种状态：

- **continue**：当前任务仍可自主执行下一步动作。
- **pause**：当前任务仍然存在，但暂时不应自主继续，通常是在等待用户协作、计划时间或其他外部条件。不同暂停原因可以具有不同的恢复条件。
- **complete**：模型判断当前任务已经完成，但在 flush 前仍保留为可重新匹配的 transaction。
- **archive**：transaction 已经在 flush 中完成归档。该状态不能由模型直接选择。

模型只能决定 **continue**、**pause** 和 **complete**；**archive** 只能由 flush 产生。

主要状态变化为：

```
新建 → continue

continue → continue
continue → pause
continue → complete

pause → continue

complete → continue
complete → archive

archive → continue
```

其中：

- `pause → continue` 表示等待条件已经满足，原任务继续运行。
- 尚未 flush 的 `complete` 如果被新的无来源刺激明确匹配，应执行 `complete → continue`。
- `complete → archive` 只能由 flush 执行。
- `archive` 不表示 transaction 被删除；合法显式恢复动作，或归档前已经有效登记的一次性 Schedule run，可以按明确 transaction 身份恢复它，并执行 `archive → continue`。旧执行批次的 Feedback 不能充当恢复动作。
- `pause` 不会因为普通 flush 而变成 `archive`。

2.1.3 Scheduled Plan 的事务逻辑

安排 Scheduled Plan 时，计划必须保存所属的 transaction 身份。该 transaction 在等待计划到期期间处于 `pause`，并持续保留其 WM 与任务状态。

计划尚未到期时，它只是未来安排，不属于当前刺激池。

计划到期后，系统生成带有原 transaction 身份的刺激并放入相应 conversation 的刺激池。该刺激被消费后：

- 不进行无来源语义匹配；
- 直接锁定原 transaction；
- 恢复原 WM 和任务状态；
- 为本次计划激活开启新的执行批次；
- 将原 transaction 激活为 `continue`；
- 沿用原 transaction 身份继续执行。

当前版本只规定一次性 Scheduled Plan：本次计划执行结束后，transaction 应进入 `complete`，并在后续 Flush 中按照普通完成任务归档。周期性计划及其重复激活状态线不属于当前版本的主线范围。

如果 transaction 在计划到期前被其他合法刺激恢复，计划本身不会因此消失。到期时若原 transaction 已有有效执行批次，Scheduled Plan 不得抢占或并行开启第二个批次，而应等待安全的批次边界；人工暂停不能被自动计划绕过，已经删除的 transaction 不能被计划

复活。若预先登记的一次性计划仍然有效，而 transaction 已经 complete 或 archive，则该计划可以作为明确来源恢复原 transaction 并开启新批次。

一次性计划已经开始执行后，如果用户在计划刺激仍处于 Thinking 时手动暂停 transaction，该刺激必须被中止且不重新入池；如果本次 Thinking 已经提交、系统正在等待异步 Feedback，则该 delivery 已经消费，不能被倒改为 aborted。两种情况下，当前执行批次与未决 Feedback 都随之失效，但“暂停当前执行”不等于“取消原计划”。尚未完成的 plan run 保持为被人工暂停阻塞，用户以后恢复 transaction 时也不能立刻抢占新的执行批次，只能在安全批次边界基于同一个 run 身份生成一次新的 delivery。这个新 delivery 是耐久计划的后续尝试，不是旧刺激重入。如果用户删除 transaction，则该 run 必须取消，不能再恢复或复活 transaction。

如果计划执行正在等待本批 Feedback，而新的无来源刺激或合法显式恢复先开启了新的执行批次，旧批次及其委托必须失效，已经消费的旧 delivery 保持原记录；plan run 解除与旧批次的绑定并进入阻塞，等新的执行批次到达安全边界后才能产生下一代 delivery。它不能永久悬挂在已经失效的批次上，也不能与新批次并行执行。

如果计划执行是因为等待用户协作而暂停，则本次执行批次正常结束，plan run 保持进行中但暂时不绑定有效批次。相关用户输入或合法显式恢复到达后，系统开启新批次并把同一个 run 重新绑定到该批次；用户输入本身已经构成恢复刺激，不应再额外制造一个 Scheduled Plan 刺激。

同一 transaction 即使登记了多个一次性计划，在任一时刻也最多只能有一个 run 处于已经 claim、尚未完成的执行中状态。一个 run 正在等待用户协作而暂时没有有效批次时，其他 run 到期仍必须等待；不能因为“当前没有 activation”就覆盖、遗忘或并行执行前一个 run。

当前版本不允许一个正在执行的 plan run 再为同一 transaction 登记下一 run 并转入 `pause(scheduled_wait)`；这种链式安排本质上已经进入重复/周期性计划状态线，统一留待后续版本设计，不能在当前一次性计划机制中隐式实现。

同一个 plan run 在任何时刻最多只能关联一个有效执行批次；同一 delivery 在同一时刻最多只有一个有效 claim。若消费者在最终 disposition 前失效，系统可以让新的消费者以新的所有权世代重新 claim 同一 delivery 身份；旧消费者此后不能再提交。delivery 一旦形成最终 disposition 就不能再次 claim，也不能改写为另一结果。暂停后由 plan run 发起的新尝试必须使用新的 delivery 身份，计划 run 最终正常完成和消费也只能发生一次。

2.1.4 Feedback 的事务逻辑

Feedback 携带明确的 transaction 来源，因此不参与无来源刺激的语义匹配，也不能因此创建新 transaction。但是，只有 transaction 身份并不足以证明 Feedback 仍然有效。

有效 Feedback 必须同时满足：

- 指向正确且仍然存在的 transaction；
- 属于该 transaction 当前有效的执行批次；
- 对应本执行批次中仍然有效的工具委托；
- 尚未被重复消费。

也就是说，Feedback 至少需要通过 `transaction_id + activation_id + delegate_id` 所表达的因果关系完成校验。有效 Feedback 只更新它所指向的 transaction，不能修改同一 conversation 中的其他 transaction。

如果 UI 手动暂停或删除 transaction，当前执行批次随之失效，其尚未返回的 Feedback 永久作废。此类 Feedback 在感知入口即被明确拒绝，不进入正常刺激池，也不能因为 transaction 后来重新激活而恢复有效。

为了等待本批 Feedback 而暂停思考时，当前执行批次仍然有效，合法 Feedback 在同一批次中继续。如果此时有新的无来源刺激先匹配到该 pause transaction，则新刺激的恢复动作会原子终止旧批次及其未决委托、开启新批次；旧 Feedback 从此永久无效。已经合法入池、但在消费前因暂停或删除而变陈旧的 Feedback，也必须在消费前终止，不能产生领域效果。

2.1.5 UI 手动暂停与删除逻辑

UI 手动暂停和删除是用户对 Transaction Runtime 的直接控制，不是需要排队等待当前思考结束和普通刺激。

手动暂停 transaction 时：

- 停止该 transaction 当前的思考过程；
- 当前正在处理的刺激明确标记为被用户中止，不重新放回刺激池；
- transaction 进入带有人工保持含义的 `pause`；
- 当前执行批次失效；
- 本批次所有尚未返回的 Feedback 永久作废。

如果当前批次来自尚未完成的一次性 Scheduled Plan，仍在 Thinking 中的 schedule delivery 同样终止且不重入；已经完成 Thinking 的 delivery 保持 consumed，也不重入。

原 run 进入人工阻塞，等 transaction 被用户恢复并再次到达安全批次边界后，才允许产生新的 delivery。

手动暂停不阻止新的无来源刺激进入感知池。无来源刺激仍可将该 transaction 作为合法的 `pause` 候选；如果匹配成功，则使用原 transaction 身份、恢复原 WM 与任务状态、开启新的执行批次，并执行 `pause` → `continue`。新的执行批次可以接收自己产生的 Feedback，旧批次 Feedback 仍然无效。

删除 transaction 时：

- 停止其当前执行并使当前执行批次失效；
- 永久拒绝后续指向该 transaction 的 Feedback；
- 不再允许该 transaction 参加无来源刺激匹配或恢复；
- 删除不等同于 `archive`，也不应悄悄创建一个替代 transaction。
- 所有仍未消费的一次性 plan run 同时取消，后续到期或重放不能复活该 transaction。

删除 transaction 不会反向改写已经发生的 Scene；若用户需要连同历史记录一并清除，应由独立的隐私清除语义负责。

2.1.6 Archive 的恢复逻辑

`archive` 是可恢复的历史状态，不是删除状态。

当合法显式恢复动作，或归档前已经登记且仍有效的一次性 Schedule run 携带明确 transaction 身份时，即使目标 transaction 已经归档，系统仍应：

- 找到原 transaction；
- 恢复它的 WM 与任务状态；
- 保留原 transaction 身份；
- 开启新的执行批次；
- 将其恢复为 `continue`；

- 继续写入原 conversation 的 Scene。

Archive transaction 不参加无来源刺激的自动语义匹配，只能通过上述明确且可校验的来源恢复。如果指定 transaction 在当前记录和归档记录中都不存在，系统应明确拒绝，不能悄悄新建 transaction 代替它。

当前版本只规定明确 transaction 身份下的 archive 恢复；通过自然语言询问主题、检索历史 transaction 再恢复的流程留待后续版本。

2.1.7 Scene 的连续逻辑

每个 conversation 只有一条 Scene 情景线。

同一 conversation 中的多个 transaction 可以交错产生行为，但这些行为都按照实际发生顺序进入同一条 Scene。能够确定所属 transaction 的 Scene 内容，应保持相应的 transaction 关联。

Transaction 的状态变化不改变 Scene 的连续性：

- transaction pause 后，既有 Scene 仍然保留；
- transaction 再次激活后，继续写入同一条 Scene；
- transaction complete 或 archive 后，既有 Scene 不被删除；
- archive transaction 按明确身份恢复后，仍回到原 conversation 的 Scene 中继续活动。

因此，Transaction 线负责保持各项任务自身的连续性与隔离性，Scene 线负责保持整个 conversation 的时间连续性。

2.1.8 Flush 的领域逻辑

Flush 是用户可感知的会话语义边界，可以由用户主动触发，也可以由系统按照明确规则自动触发。它负责结束当前连续交互段、巩固本段 Scene，并把符合条件的 complete transaction 标记为 archive。自动 Flush 同样必须留下对用户和系统均可观察的边界，不能表现为不可见的后台清理。

在 flush 发生前，complete transaction 仍然可以参与无来源刺激匹配；匹配成功后，它重新进入 continue，不应再被本次 flush 归档。

处于 pause 或 continue 的 transaction 不能被普通 flush 归档。以未来激活为目的而保持 pause 的 Scheduled Plan transaction 同样不能被归档。

Flush 必须保证 Scene 的处理边界与 transaction 的归档结果在逻辑上保持一致，不能出现“情景线已经越过该任务，但任务仍未归档”或者“任务已经归档，但相应情景尚未完成处理”的矛盾状态。

Flush 以后，已经 archive 的 transaction 不再参与无来源自动匹配。同一句自然语言在 Flush 前后得到不同的候选范围，是该用户可见会话边界的预期结果，不属于系统错误。当前版本若没有明确 transaction 身份，不负责通过自然语言自动检索 archive。

2.2 Conversation 内感知刺激池的运行逻辑

每个 conversation 拥有独立的刺激池。刺激只在本 conversation 内排序和消费，不与其他 conversation 的刺激比较优先级。

当刺激实际到达时，刺激池保存：

- 刺激自身的唯一身份；
- 刺激类别和内容；
- 所属 conversation；

- 优先级；
- 已经存在的 transaction 来源信息；
- 对 Feedback 而言，所属执行批次与工具委托信息。

刺激池必须原样保留来源信息，但不自行判断或修改 transaction 归属。

池中刺激按照以下规则消费：

1. 优先级更高的刺激先被选择。
2. 相同优先级的刺激按照进入当前 conversation 刺激池的先后顺序处理。
3. 同一个 conversation 每次只消费一个刺激。
4. 已经消费的刺激不会再次作为新的未知刺激被取出。
5. 正在处理一个刺激时，后来到达的刺激可以继续进入池中。
6. 当前刺激处理完毕并到达下一次选择边界后，重新从池内现有刺激中选择优先级最高的一条。
7. 池中刺激消费完毕后，本轮消费结束；之后出现的新刺激应能够重新触发消费。

刺激一旦被系统接纳，在形成 consumed、aborted、预期废弃或其他明确最终处置以前，不能因为进程重启、消费者切换或暂时故障而静默丢失，也不能被当作两个独立刺激重复处理。同一 conversation 在同一时刻只能有一个有效的 Thinking 消费所有者；所有权发生转移后，旧消费者即使恢复也不能再提交本次刺激的 Transaction 变化或工具委托。

一次刺激的消费边界是：思考层已经处理本次刺激，完成必要的 Transaction 状态更新，并在需要时发出异步工具委托。刺激消费者不等待工具实际执行完成；工具结果以后以新的 Feedback 刺激进入感知层。因此，“逐个消费”只串行化刺激对应的思考和状态提交，不把整个工具执行生命周期锁在一个消费者中。

异步执行不能被描述成系统天然拥有跨服务的“绝对一次”能力。只有工具明确支持同一调用身份的幂等去重时，系统才能承诺一次可见效果；不支持时，系统必须明确采用“最多尝试一次”或“至少尝试一次”的策略，并把无法确认是否已执行的结果标记为 `uncertain`，让用户和系统后续流程可见，不能把不确定性伪装成成功。

其中“至少尝试一次”只在目标最终恢复可达的前提下保证至少一次外部可见效果。当前版本不定义独立的外部 effect 取消语义；UI Pause/Delete 只控制 Transaction 执行与后续 Feedback 接纳，不能被解释为已经撤销可能在途的外部 effect。

Feedback 在进入刺激池前必须完成执行批次与委托有效性校验。被手动暂停、删除或旧执行批次作废的 Feedback 属于预期拒绝，不进入正常池。UI 手动暂停和删除属于直接控制动作，同样不在普通刺激池中等待调度。

工具结果无论最终被接纳为 Feedback，还是因为对应批次已经失效而得到预期拒绝，都必须形成一个可观察的最终交付结果；后者不能在后台被无限重复投递。

未来刺激在真正发生前不属于任何当前刺激池。例如 Scheduled Plan 在到期前只是计划；到期后生成的刺激才作为实际刺激进入其 conversation 的池中。

刺激离开池后才进入 transaction 路由：

- 已有明确且通过来源有效性校验的刺激直接锁定指定 transaction。
- 没有明确来源的刺激进入无来源归因流程。

刺激池本身不决定 transaction 的创建、恢复或状态变化。

2.3 无来源刺激归因的运行逻辑

2.3.1 首先判断来源及其有效性

Transaction 归属的第一判断依据不是刺激的具体类型，而是刺激是否已经携带明确的 transaction 身份。具有明确来源的刺激不进行无来源语义匹配，但“无需匹配”不等于“无需校验”。

不同来源遵循不同的有效性逻辑：

- 到期 Scheduled Plan 在计划记录有效时定位原 transaction。正常等待主线开启新批次并执行 `pause` → `continue`；若已有有效批次则等待安全边界，若处于人工暂停则保持等待，若已删除则取消，若已经 `complete/archive` 则可由这条预先登记的明确来源恢复。
- 显式 UI 恢复或其他合法恢复动作可以定位 `pause`、尚未 flush 的 `complete` 或 `archive` transaction，恢复原 WM 与任务状态，并开启新的执行批次。
- Feedback 必须同时匹配原 transaction、当前有效执行批次和有效工具委托；它只能继续本批次因果链，不能仅凭 transaction 身份唤醒人工暂停、`archive` 或已经删除的 transaction。
- 指向不存在、已删除或无效执行批次的刺激应明确拒绝，不能悄悄新建 transaction 代替。

被允许重试并重新放入刺激池、而且已经确定 transaction 归属的刺激，继续保持原 transaction 归属，不能再次作为无来源刺激匹配。UI 手动暂停所中止的当前刺激不属于这种重试，不重新进入刺激池。

2.3.2 无来源刺激的候选范围

只有没有 transaction 身份的刺激才进入语义匹配。

候选 transaction 必须同时满足：

- 属于当前 conversation；
- 状态为 `pause`，或者状态为尚未 flush 的 `complete`。

以下 transaction 不参加无来源刺激匹配：

- 当前处于 `continue` 的 transaction；
- 已经 `archive` 的 transaction；
- 已经由用户删除的 transaction；
- 其他 conversation 的 transaction；
- 已经完成 flush 的历史 transaction。

候选范围先由 conversation 边界和 transaction 状态决定；语义匹配只能在这个合法范围内选择。

2.3.3 匹配或新建

无来源刺激与候选 transaction 进行语义比较时，需要判断新刺激是否在自然延续某个候选任务的目标、上下文和当前状态。

归因结果只有两种：

锁定一个已有 transaction
或
新建一个 transaction

具体规则为：

- 如果能够明确匹配唯一候选，则锁定该 transaction。

- 匹配到 `pause` 时，恢复其 WM 和任务状态，开启新的执行批次，并执行 `pause → continue`。即使该 transaction 由 UI 手动暂停，无来源刺激也可以通过合法匹配恢复它；“人工暂停后拒绝旧 Feedback”的规则不会阻止新的无来源刺激。
- 匹配到尚未 flush 的 `complete` 时，恢复其 WM 和任务状态，开启新的执行批次，并执行 `complete → continue`。
- 如果没有匹配候选，则新建 transaction，并以 `continue` 开始。
- 如果多个候选之间无法明确判断，则不任意选择，直接新建 transaction。
- 新建 transaction 时，原有候选不得发生状态或内容变化。

一个刺激最多只能锁定一个 transaction，不能同时更新多个 transaction。

2.3.4 归属稳定性

刺激一旦完成归因，其 transaction 归属在后续处理过程中必须保持稳定。

后续执行、允许重试的暂时中断、重新进入刺激池、Scene 记录以及再次恢复，都应继续使用已经确定的 transaction 身份，不能再次把它当作无来源刺激重新匹配。

归属稳定指 transaction 身份保持稳定，不代表失效的执行批次可以继续使用。Transaction 被手动暂停、恢复并开启新执行批次后，后续处理使用原 transaction 身份和新的执行批次身份；旧批次 Feedback 仍然无效。

2.4 三大功能的端到端逻辑

三大功能共同形成以下稳定的领域运行主线：

刺激实际到达

- Feedback 先校验 transaction、执行批次和工具委托
- 无效 Feedback 在感知入口明确拒绝
- 合法刺激进入所属 conversation 的刺激池
- 按 conversation 内优先级逐个消费
- 判断是否已有 transaction 来源
- 有来源则按来源类型完成确定性校验并锁定原 transaction
- 无来源则在 `pause` 和未 `flush complete` 中匹配
- 明确匹配则继续原 transaction，并在需要时开启新执行批次
- 无匹配或匹配模糊则新建 transaction
- 思考层针对本次刺激推进 WM 和任务状态
- 如需使用工具，则发出带执行批次和委托身份的异步执行
- 本次刺激消费结束，不等待工具结果
- 工具结果作为 Feedback 重新经过感知入口
- 将实际发生的过程持续写入 conversation 的 Scene
- transaction 最终进入 `continue`、`pause` 或 `complete`
- 用户可见的 Flush 收束本场会话，并将符合条件的 `complete` 标记为 `archive`

用户控制形成一条不等待普通刺激调度的旁路：

UI Pause

- 中止当前思考，当前刺激不重新入池
- 当前执行批次失效
- **transaction** 进入 **pause**
- 旧批次 **Feedback** 永久无效

无来源刺激明确匹配该 **pause transaction**

- 保留原 **transaction** 身份
- 开启新执行批次
- **transaction** 恢复 **continue**

UI Delete

- **transaction** 生命周期终止
- 不再参加匹配或恢复
- 后续关联 **Feedback** 永久拒绝

这条主线是架构迁移前后共同的语义基准。具体 Runtime 可以改变内部结构，但不能改变三大功能各自回答的问题、彼此之间的职责边界，以及上述领域行为。